Lab_Assignment_2.1

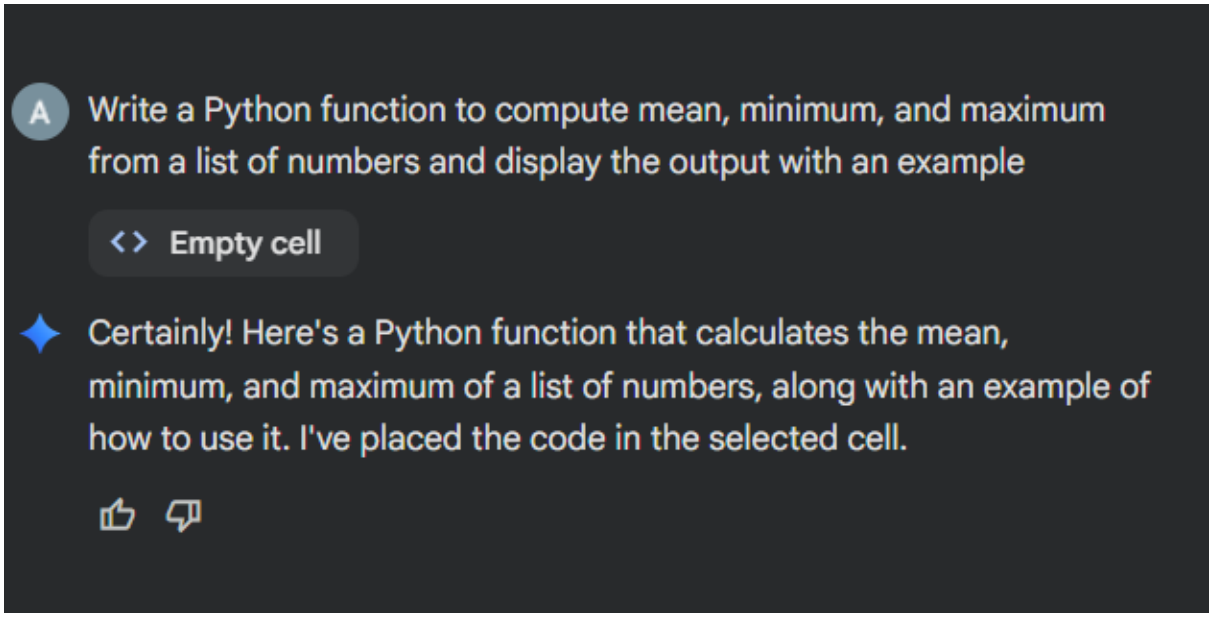
P.HARSHAVARDHAN
2303A53038
BATCH - 46

Task 1: Statistical Summary for Survey Data

❖ Scenario:You are a data analyst intern working with survey responses stored asnumerical lists.

❖ Task:Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values.

INPUT:



CODE:

```python
import statistics
def calculate_stats(numbers):
    """
    Calculates the mean, minimum, and maximum of a list of numbers.

    Args:
        numbers (list): A list of numerical values.

    Returns:
        dict: A dictionary containing 'mean', 'min', and 'max' values.
              Returns None for mean, min, and max if the list is empty.
    """
    if not numbers:
        return {
            'mean': None,
            'min': None,
            'max': None
        }
    mean_value = statistics.mean(numbers)
    min_value = min(numbers)
    max_value = max(numbers)
    return {
        'mean': mean_value,
        'min': min_value,
        'max': max_value
    }
# Example usage:
my_list = [10, 20, 30, 40, 50, 15, 25]
results = calculate_stats(my_list)

print(f"Original list: {my_list}")
print(f"Mean: {results['mean']}")
print(f"Minimum: {results['min']}")
print(f"Maximum: {results['max']}")

# Example with an empty list
empty_list = []
empty_results = calculate_stats(empty_list)
print(f"\nOriginal list: {empty_list}")
print(f"Mean: {empty_results['mean']}")
print(f"Minimum: {empty_results['min']}")
print(f"Maximum: {empty_results['max']}")
```

OUTPUT:

```
Original list: [10, 20, 30, 40, 50, 15, 25]
Mean: 27.142857142857142
Minimum: 10
Maximum: 50

Original list: []
Mean: None
Minimum: None
Maximum: None
```

Function to calculate statistical summary of survey data

Calculate the mean value of the list

Find the minimum value in the list

Find the maximum value in the list

Return mean, minimum, and maximum values

Sample list of survey responses

Call the function

Display the results

Task 2: Armstrong Number – AI Comparison

Scenario: You are evaluating AI tools for numeric validation logic.

Task:Generate an Armstrong number checker using Gemini and GitHub Copilot.

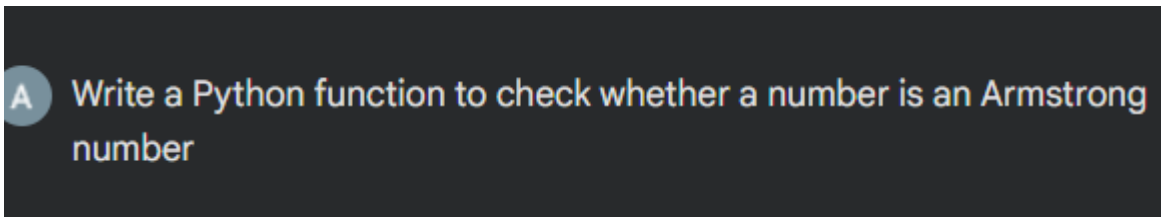
Compare their outputs, logic style, and clarity.

❖ Expected Output:

➢ Side-by-side comparison table

➢ Screenshots of prompts and generated code

INPUT PROMPT:



CODE:

```python
def is_armstrong_number(number):
    """
    Checks if a given number is an Armstrong number.

    An Armstrong number is a number that is equal to the sum of its own digits
    each raised to the power of the number of digits.
    For example, 153 is an Armstrong number because 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153.

    Args:
        number (int): The number to check.

    Returns:
        bool: True if the number is an Armstrong number, False otherwise.
    """
    if not isinstance(number, int) or number < 0:
        raise ValueError("Input must be a non-negative integer.")

    # Convert number to string to get its digits and count them
    num_str = str(number)
    num_digits = len(num_str)

    sum_of_powers = 0
    for digit_char in num_str:
        digit = int(digit_char)
        sum_of_powers += digit ** num_digits

    return sum_of_powers == number

# Example usage:
print(f"Is 153 an Armstrong number? {is_armstrong_number(153)}")
print(f"Is 370 an Armstrong number? {is_armstrong_number(370)}")
print(f"Is 9474 an Armstrong number? {is_armstrong_number(9474)}")
print(f"Is 100 an Armstrong number? {is_armstrong_number(100)}")
print(f"Is 9 an Armstrong number? {is_armstrong_number(9)}")

# Example with a larger number (if it's an Armstrong number)
# print(f"Is 54748 an Armstrong number? {is_armstrong_number(54748)}") # Not an Armstrong number
# print(f"Is 1634 an Armstrong number? {is_armstrong_number(1634)}") # Is an Armstrong number
```

OUTPUT:

```
Is 153 an Armstrong number? True
Is 370 an Armstrong number? True
Is 9474 an Armstrong number? True
Is 100 an Armstrong number? False
Is 9 an Armstrong number? True
```

Function to check whether a number is an Armstrong number

Convert the number into individual digits

Count the number of digits

Calculate the sum of each digit raised to the power of total digits

Compare the calculated sum with the original number

Display whether the number is Armstrong or not

Take input from the user

Call the function and show the result

Task 3: Leap Year Validation Using Cursor AI

Scenario:You are validating a calendar module for a backend system.

❖ Task:Use Cursor AI to generate a Python program that checks whether a given year is a leap year.

Use at least two different prompts and observe changes in code.

❖ Expected Output:

➢ Two versions of code

➢ Sample inputs/outputs

➢ Brief comparison

INPUT:

```python
import statistics
def calculate_stats(numbers):
    """
    Calculates the mean, minimum, and maximum of a list of numbers.

    Args:
        numbers (list): A list of numerical values.

    Returns:
        dict: A dictionary containing 'mean', 'min', and 'max' values.
              Returns None for mean, min, and max if the list is empty.
    """
    if not numbers:
        return {
            'mean': None,
            'min': None,
            'max': None
        }
    mean_value = statistics.mean(numbers)
    min_value = min(numbers)
    max_value = max(numbers)
    return {
        'mean': mean_value,
        'min': min_value,
        'max': max_value
    }
# Example usage:
my_list = [10, 20, 30, 40, 50, 15, 25]
results = calculate_stats(my_list)

print(f"Original list: {my_list}")
print(f"Mean: {results['mean']}")
print(f"Minimum: {results['min']}")
print(f"Maximum: {results['max']}")

# Example with an empty list
empty_list = []
empty_results = calculate_stats(empty_list)
print(f"\nOriginal list: {empty_list}")
print(f"Mean: {empty_results['mean']}")
print(f"Minimum: {empty_results['min']}")
print(f"Maximum: {empty_results['max']}")
```

OUTPUT:

```
Problems   Output   Debug Console   Terminal   Ports

PS C:\Users\anagandula gouthami\OneDrive\Desktop\AI Coding>
eDrive/Desktop/AI Coding/lab-2.1.py"
Enter a year: 2005
2005 is not a leap year
Enter a year:
```

COMMENTS:

Program to check whether a given year is a leap year

Take year input from the user

Check if the year is divisible by 400

Check if the year is divisible by 100

Check if the year is divisible by 4

Display whether the year is a leap year or not

Function to validate leap year logic

Call the function and print the result

Task 4: Student Logic + AI Refactoring (Odd/Even Sum)

❖ Scenario:Company policy requires developers to write logic before using AI.

❖ Task:Write a Python program that calculates the sum of odd and even numbers

in a tuple, then refactor it using any AI tool.

❖ Expected Output:

➢ Original code

➢ Refactored code

➢ Explanation of improvements

INPUT:

```python
# Write a Python program to calculate the sum of odd numbers and even numbers in a tuple and display the results.
numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
sum_odd = 0
sum_even = 0
for num in numbers:

    if num % 2 == 0:
        sum_even += num
    else:
        sum_odd += num
print("Sum of odd numbers:", sum_odd)
print("Sum of even numbers:", sum_even)
```

OUTPUT:

```
Sum of odd numbers: 25
Sum of even numbers: 30
```

COMMENTS:

Calculate sum of even and odd numbers

Loop through the tuple

Check even or odd

Add values accordingly

Print the results

Refactored using sum()

Code is shorter and clearer